TABLEAU & MATRICE

1. Tableau statique :

- **Définition :**

Un **tableau statique** est un **ensemble d'éléments du même type** dont la taille est **fixée à la compilation**.

- **Explication :**

- Tous les éléments sont **contigus en mémoire**.
- La taille **ne peut pas changer** pendant l'exécution du programme.
- Les indices commencent à **0**.

- **Exemple :**

```
#include <stdio.h>

int main() {

    int notes[5] = {10, 12, 15, 18, 20};

// tableau statique de 5 entiers

    for(int i = 0; i < 5; i++) {
```

```
        printf("Note %d: %d\n", i+1, notes[i]);
    }
    return 0;
}
```

2. Tableau dynamique :

- Définition :

Un **tableau dynamique** est un **tableau dont la taille peut être déterminée à l'exécution**, et la mémoire est **allouée dynamiquement**.

- Explication :
  - Utilise les fonctions de <stdlib.h> : malloc(), calloc(), free().
  - Permet d'économiser de la mémoire et d'adapter la taille selon les besoins.
- Exemple :

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int n;
    printf("Entrez le nombre d'éléments: ");
```

```
    scanf("%d", &n);
    int *tableau = (int *) malloc(n * sizeof(int)); // allocation dynamique
    for(int i = 0; i < n; i++) {
        tableau[i] = i + 1; // remplir le tableau
    }
    printf("Éléments du tableau: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", tableau[i]);
    }
    free(tableau); // libérer la mémoire
    return 0;
}
```

3. Matrice (tableau à deux dimensions) :

- Définition :

Une **matrice** est un **tableau à deux dimensions**, c'est-à-dire un tableau de tableaux, utilisé pour représenter des données organisées en **lignes et colonnes**.

- Explication :

- La mémoire peut être **statique** (taille fixe) ou **dynamique** (allocation à l'exécution).
- Pour accéder à un élément : matrice[i][j] (i = ligne, j = colonne).

- **Exemple :**

```
#include <stdio.h>

int main() {
    int matrice[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            printf("%d ", matrice[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

4. Matrice dynamique :

- Explication :
  - Utilise malloc() pour chaque ligne ou pour l'ensemble des données.
  - Permet de créer une **matrice de taille flexible**.
- Exemple :

```
#include <stdio.h>

#include <stdlib.h>

int main() {

   int lignes = 2, colonnes = 3;

   int **matrice = (int **) malloc(lignes * sizeof(int *));

   for(int i = 0; i < lignes; i++) {

      matrice[i] = (int *) malloc(colonnes * sizeof(int));

   }

   // remplir la matrice

   int compteur = 1;

   for(int i = 0; i < lignes; i++) {

      for(int j = 0; j < colonnes; j++) {

         matrice[i][j] = compteur++;

      }

   }
```

```
    // afficher la matrice
    for(int i = 0; i < lignes; i++) {
        for(int j = 0; j < colonnes; j++) {
            printf("%d ", matrice[i][j]);
        }
        printf("\n");
    }
    // libérer la mémoire
    for(int i = 0; i < lignes; i++) {
        free(matrice[i]);
    }
    free(matrice);
    return 0;
}
```